



## รายงาน PS02: Process Creation & Pipes

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

|                           |             |
|---------------------------|-------------|
| นายภูมิพัฒน์ อภิวัตนะพงศ์ | 67070501035 |
| นายวิรัช ทองอุทัยศรี      | 67070501041 |
| นายเจษฎา เกียรติกมลวงค์   | 67070501080 |

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems  
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์  
ภาคการศึกษาที่ 1 ปีการศึกษา 2569  
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

# Contents

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>สภาพแวดล้อมที่ใช้ในการทดลอง</b>                        | <b>2</b>  |
| <b>1 fork() แบบมีและไม่มี wait()</b>                      | <b>3</b>  |
| 1.1 Source code . . . . .                                 | 3         |
| 1.1.1 1.1 โปรแกรม helloworld (ไม่มี wait()) . . . . .     | 3         |
| 1.1.2 1.2 โปรแกรมที่เพิ่ม wait() . . . . .                | 4         |
| 1.2 ผลการทดลอง . . . . .                                  | 5         |
| 1.3 อภิปรายผล . . . . .                                   | 5         |
| <b>2 Process Status และ sleep()</b>                       | <b>6</b>  |
| 2.1 Source code . . . . .                                 | 6         |
| 2.1.1 2.1 Child ตายก่อน โดย Parent ยังทำงานอยู่ . . . . . | 6         |
| 2.1.2 2.2 Parent ตายก่อน โดย Child ยังทำงานอยู่ . . . . . | 7         |
| 2.2 ผลการทดลอง . . . . .                                  | 8         |
| 2.2.1 2.1 Child ตายแล้ว แต่ Parent ยังทำงานอยู่ . . . . . | 8         |
| 2.2.2 2.2 Parent ตายแล้ว แต่ Child ยังทำงานอยู่ . . . . . | 9         |
| 2.3 อภิปรายผล . . . . .                                   | 10        |
| <b>3 จำนวน process สูงสุดที่ fork ได้</b>                 | <b>11</b> |
| 3.1 Source code . . . . .                                 | 11        |
| 3.1.1 ส่วนหลัก: ฟังก์ชัน chain() และการรายงานผล . . . . . | 11        |
| 3.1.2 ส่วน main(): กำหนดเพดานและรับผลลัพธ์ . . . . .      | 13        |
| 3.2 ผลการทดลอง . . . . .                                  | 15        |
| 3.2.1 3.1 การวัดครั้งแรก . . . . .                        | 15        |
| 3.2.2 3.2 เมื่อมีโปรแกรมอื่นทำงานค้างอยู่ . . . . .       | 15        |
| 3.3 อภิปรายผล . . . . .                                   | 16        |
| <b>4 การสื่อสารผ่าน Pipe</b>                              | <b>17</b> |
| 4.1 Source code . . . . .                                 | 17        |
| 4.2 ผลการทดลอง . . . . .                                  | 19        |
| 4.3 อภิปรายผล . . . . .                                   | 19        |
| <b>5 การทดลองกรณีพิเศษของ Pipe</b>                        | <b>21</b> |
| 5.1 5.1 ผู้ส่งพยายามอ่านข้อความของตัวเองกลับ . . . . .    | 21        |
| 5.2 5.2 ผู้รับอ่านก่อนที่ผู้ส่งจะส่ง . . . . .            | 22        |
| 5.3 5.3 ผู้ส่งส่งหลายข้อความก่อนที่ผู้รับจะอ่าน . . . . . | 23        |
| 5.4 สรุปผลการทดลองข้อ 5 . . . . .                         | 25        |
| 5.5 อภิปรายผล . . . . .                                   | 25        |

## สภาพแวดล้อมที่ใช้ในการทดลอง

---

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ระบบปฏิบัติการ | Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11)                                                                                                                                                                                                                                                                                                                                                                                          |
| Kernel         | 6.6.87.2-microsoft-standard-WSL2 (ตรวจด้วย <code>uname -r</code> )                                                                                                                                                                                                                                                                                                                                                                 |
| คอมไพเลอร์     | gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0                                                                                                                                                                                                                                                                                                                                                                                        |
| คำสั่งคอมไพล์  | <pre>gcc -Wall -Wextra -o q1_1 q1_1_helloworld.c gcc -Wall -Wextra -o q1_2 q1_2_wait.c gcc -Wall -Wextra -o q2_1 q2_1_child_first.c gcc -Wall -Wextra -o q2_2 q2_2_parent_first.c gcc -Wall -Wextra -o q3_forkcount q3_forkcount.c gcc -Wall -Wextra -o q4_pipe q4_pipe.c gcc -Wall -Wextra -o q5_1 q5_1_sender_reads_own.c gcc -Wall -Wextra -o q5_2 q5_2_read_before_send.c gcc -Wall -Wextra -o q5_3 q5_3_many_messages.c</pre> |
| ผลการคอมไพล์   | ผ่านทุกไฟล์ ไม่มี warning                                                                                                                                                                                                                                                                                                                                                                                                          |

---

**หมายเหตุเรื่อง WSL2** WSL2 รัน Linux kernel จริง ดังนั้น `fork()`, `wait()`, `pipe()` และคำสั่ง `ps` ทำงานได้ตามปกติ อย่างไรก็ตามตัวเลขขีดจำกัดจำนวน process ในข้อ 3 อาจต่างจาก Linux ที่ติดตั้งแบบ native หรือแบบ VM เพราะ WSL2 จัดสรรหน่วยความจำให้ VM แบบไดนามิก

# 1 fork() แบบมีและไม่มี wait()

## 1.1 Source code

### 1.1.1 1.1 โปรแกรม helloworld (ไม่มี wait())

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     printf("Hello World!\n");
8
9     pid_t pid = fork();
10    if (pid < 0) {
11        perror("fork");
12        exit(EXIT_FAILURE);
13    }
14
15    if (pid == 0) {
16        /* child */
17        printf("I am the child, my PID = %d, my parent PID = %d\n",
18              getpid(), getppid());
19    } else {
20        /* parent */
21        printf("I am the parent, my PID = %d, my child PID = %d\n",
22              getpid(), pid);
23    }
24
25    printf("Common line printed by PID %d\n", getpid());
26    return 0;
27 }
```

Code 1: q1\_1\_helloworld.c – fork() without wait()

**คำอธิบายการทำงาน:** เรียก fork() เพื่อสร้าง child process หากเป็น child (pid == 0) จะพิมพ์ PID ของตนเองและ parent ส่วน parent จะพิมพ์ PID ของตนเองและลูก เนื่องจากไม่มี wait() parent จึงจบการทำงานโดยไม่รอ child

### 1.1.2 1.2 โปรแกรมที่เพิ่ม wait()

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main(void)
7  {
8      printf("Hello World!\n");
9
10     pid_t pid = fork();
11     if (pid < 0) {
12         perror("fork");
13         exit(EXIT_FAILURE);
14     }
15
16     if (pid == 0) {
17         /* child */
18         printf("I am the child, my PID = %d, my parent PID = %d\n",
19             getpid(), getppid());
20     } else {
21         /* parent */
22         wait(NULL);
23         printf("I am the parent, my PID = %d, my child PID = %d\n",
24             getpid(), pid);
25     }
26
27     printf("Common line printed by PID %d\n", getpid());
28     return 0;
29 }

```

Code 2: q1\_2\_wait.c – fork() with wait()

**คำอธิบายการทำงาน:** เพิ่มการเรียก wait(NULL) ในฝั่ง parent เพื่อสั่งให้ parent หยุดรอ (Blocked) จนกว่า child จะทำงานเสร็จและคืนสถานะเรียบร้อย บังคับให้ลำดับการแสดงผลของ child เกิดขึ้นก่อน parent เสมอ

## 1.2 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q1_1 q1_1_helloworld.c
$ ./q1_1
Hello World!
I am the parent, my PID = 3642, my child PID = 3643
Common line printed by PID 3642
I am the child, my PID = 3643, my parent PID = 3642
Common line printed by PID 3643

$ ./q1_1
Hello World!
I am the parent, my PID = 3644, my child PID = 3645
Common line printed by PID 3644
I am the child, my PID = 3645, my parent PID = 3632    <-- PPID changed (Orphan)!
Common line printed by PID 3645
```

Code 3: ผลลัพธ์ของข้อ 1.1 (ไม่มี wait())

```
$ gcc -Wall -Wextra -o q1_2 q1_2_wait.c
$ ./q1_2
Hello World!
I am the child, my PID = 3649, my parent PID = 3648
Common line printed by PID 3649
I am the parent, my PID = 3648, my child PID = 3649
Common line printed by PID 3648
```

Code 4: ผลลัพธ์ของข้อ 1.2 (มี wait())

## 1.3 อภิปรายผล

จากการทดลองเปรียบเทียบระหว่างโปรแกรมข้อ 1.1 (ไม่มี wait()) และข้อ 1.2 (มี wait()) สรุปความแตกต่างที่สำคัญได้ดังนี้

- **การชิงโครโนซ์และลำดับ Output:** ในข้อ 1.1 ทั้ง Parent และ Child แย่งกันทำงานอย่างอิสระ ลำดับการพิมพ์ข้อความจึงขึ้นอยู่กับ Kernel Scheduler และไม่แน่นอน (Non-deterministic) ในขณะที่ข้อ 1.2 การเรียก wait(NULL) จะทำให้ Parent หยุดรอ (Blocked) จนกว่า Child จะทำงานเสร็จ ลำดับ Output จึงมีความแน่นอน (Deterministic) โดยข้อความของ Child จะปรากฏก่อนเสมอ
- **ปรากฏการณ์ Orphan Process:** ในข้อ 1.1 มีโอกาสที่ Parent จะทำงานเสร็จและจบโปรแกรมไปก่อน ทำให้ Child กลายเป็น Orphan Process และถูกเคอร์เนลยกให้ init (หรือ Subreaper) เป็น Parent ใหม่แทน (สังเกตได้จาก PPID ของ Child ที่เปลี่ยนไปในการรันครั้งที่ 2) ส่วนในข้อ 1.2 จะไม่เกิดเหตุการณ์นี้เนื่องจาก Parent รอ Child อยู่เสมอ

## 2 Process Status และ sleep()

การทดลองนี้ใช้ `sleep()` เป็นเครื่องมือควบคุม ลำดับการตาย ของ parent และ child เพื่อสร้างสถานการณ์สองแบบที่ตรงข้ามกัน แล้วใช้คำสั่ง `ps` ตรวจสอบสถานะของ process ในช่วงเวลาที่สนใจ ทั้งสองโปรแกรมจงใจไม่เรียก `wait()` เพื่อให้เห็นพฤติกรรมที่เกิดขึ้นเมื่อ parent ไม่เก็บกวาดลูกของตนเอง

### 2.1 Source code

#### 2.1.1 2.1 Child ตายก่อน โดย Parent ยังทำงานอยู่

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  #define PARENT_LIFETIME 30  /* seconds the parent stays alive after child dies */
6
7  int main(void)
8  {
9      pid_t pid = fork();
10
11     if (pid < 0) {
12         perror("fork");
13         exit(EXIT_FAILURE);
14     }
15
16     if (pid == 0) {
17         /* CHILD: dies almost immediately */
18         printf("[child ] PID = %d, PPID = %d -- exiting right now\n",
19             getpid(), getppid());
20         fflush(stdout);
21         exit(0);
22     }
23
24     /* PARENT: outlives the child but never reaps it */
25     printf("[parent] PID = %d, child PID = %d\n", getpid(), pid);
26     printf("[parent] NOT calling wait(). Sleeping %d s.\n", PARENT_LIFETIME);
27     printf("[parent] Inspect now with: ps -ef | grep -e q2_1 -e defunct\n");
28     fflush(stdout);
29
30     sleep(PARENT_LIFETIME);
31
32     printf("[parent] PID = %d exiting; the zombie child %d is now re-parented "
33         "to init/systemd and reaped.\n", getpid(), pid);
34     return 0;
35 }

```

Code 5: q2\_1\_child\_first.c – child dies first, parent never reaps

**คำอธิบายการทำงาน:** หลัง `fork()` ฝั่ง child จะพิมพ์ PID ของตนเองแล้วเรียก `exit(0)` ทันที ส่วน parent เข้าสู่ `sleep(30)` ทำให้เกิดช่วงเวลา 30 วินาทีที่ child ตายไปแล้วแต่ parent ยังมีชีวิตอยู่และยังไม่ได้เรียก `wait()` จึงเป็นช่วงที่เปิดอีก terminal หนึ่งเข้าไปตรวจด้วย `ps` ได้ทัน การเรียก `fflush(stdout)` ก่อน `exit()` และก่อน `sleep()` เป็นการบังคับให้ข้อความถูกเขียนออกทันที ไม่ค้างอยู่ใน buffer

### 2.1.2 2.2 Parent ตายก่อน โดย Child ยังทำงานอยู่

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  #define CHILD_LIFETIME 30
6
7  int main(void)
8  {
9      pid_t pid = fork();
10
11     if (pid < 0) {
12         perror("fork");
13         exit(EXIT_FAILURE);
14     }
15
16     if (pid == 0) {
17         /* CHILD: outlives the parent */
18         printf("[child ] PID = %d, PPID = %d (original parent, still alive)\n",
19             getpid(), getppid());
20         fflush(stdout);
21
22         sleep(2); /* give the parent time to die */
23
24         printf("[child ] PID = %d, PPID = %d (parent is gone -> re-parented)\n",
25             getpid(), getppid());
26         printf("[child ] Inspect now with: ps -ef | grep q2_2\n");
27         fflush(stdout);
28
29         sleep(CHILD_LIFETIME);
30
31         printf("[child ] PID = %d exiting, final PPID = %d\n",
32             getpid(), getppid());
33         exit(0);
34     }
35
36     /* PARENT: exits immediately, without waiting */
37     printf("[parent] PID = %d, child PID = %d -- exiting immediately\n",
38         getpid(), pid);
39     fflush(stdout);
40     return 0;
41 }

```

Code 6: q2\_2\_parent\_first.c – parent dies first, child becomes an orphan

**คำอธิบายการทำงาน:** สลับบทบาทจากข้อ 2.1 คือ parent พิมพ์ข้อความแล้วจบการทำงานทันที ส่วน child อยู่ต่ออีก 30 วินาที จุดสำคัญคือ child เรียก `getppid()` สองครั้ง โดยมี `sleep(2)` คั่นกลาง ครั้งแรกพิมพ์ตอน parent ยังมีชีวิต ครั้งที่สองพิมพ์หลัง parent ตายไปแล้ว ทำให้เห็นค่า PPID ที่เปลี่ยนไปได้จากตัวโปรแกรมเองโดยไม่ต้องพึ่ง `ps`



## 2.2 ผลการทดลอง

### 2.2.1 2.1 Child ตายแล้ว แต่ Parent ยังทำงานอยู่

```
$ gcc -Wall -Wextra -o q2_1 q2_1_child_first.c
$ ./q2_1 &
[parent] PID = 6908, child PID = 6910
[parent] NOT calling wait(). Sleeping 30 s.
[child ] PID = 6910, PPID = 6908 -- exiting right now

$ ps -ef | grep -e q2_1 -e defunct | grep -v grep
UID      PID  PPID  C STIME TTY      TIME    CMD
poomipat 6908   6900  0 18:58 pts/2    00:00:00 ./q2_1
poomipat 6910   6908  0 18:58 pts/2    00:00:00 [q2_1] <defunct>

$ ps -eo pid,ppid,stat,comm | grep q2_1 | grep -v grep
  PID  PPID STAT COMMAND
  6908  6900 S+   q2_1      <- parent sleeping
  6910  6908 Z+   q2_1      <- child is a ZOMBIE

$ ps -el | grep q2_1
F S  UID      PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY      TIME CMD
0 S  1000    6908  6900  0  80   0 -   670 hrtime pts/2    00:00:00 q2_1
1 Z  1000    6910  6908  0  80   0 -    0 -      pts/2    00:00:00 q2_1
                                ^^ SZ = 0 : all memory already released
```

Code 7: ผลลัพธ์และสถานะจาก ps ในกรณี 2.1

Table 1: สรุปค่าที่สังเกตได้ในกรณี 2.1

| รายการ                    | ค่าที่สังเกตได้                               |
|---------------------------|-----------------------------------------------|
| PID ของ parent            | 6908                                          |
| PID ของ child             | 6910                                          |
| PPID ของ child            | 6908 (ยังเป็น parent ตัวจริง ไม่ถูก reparent) |
| สถานะของ child ใน ps      | Z+ แสดงผลเป็น [q2_1] <defunct>                |
| หน่วยความจำของ child (SZ) | 0                                             |

เมื่อรอจนครบ 30 วินาทีและ parent จบการทำงาน ตรวจสอบด้วย ps อีกครั้งพบว่า ไม่เหลือ process ชื่อ q2\_1 อยู่ในระบบเลย แสดงว่า zombie ถูกเก็บกวาดทันที ที่ parent ตาย

## 2.2.2 2.2 Parent ตายแล้ว แต่ Child ยังทำงานอยู่

```
$ gcc -Wall -Wextra -o q2_2 q2_2_parent_first.c
$ ./q2_2 &
[parent] PID = 6857, child PID = 6859 -- exiting immediately
[child ] PID = 6859, PPID = 6857 (original parent, still alive)
[child ] PID = 6859, PPID = 6846 (parent is gone -> re-parented)

$ ps -ef | grep q2_2 | grep -v grep
UID      PID    PPID  C STIME TTY      TIME    CMD
poomipat 6859    6846   0 18:57 pts/2    00:00:00 ./q2_2

$ ps -eo pid,ppid,stat,comm | grep q2_2 | grep -v grep
PID    PPID STAT COMMAND
6859   6846 S+   q2_2          <- normal sleeping state, NO zombie at all
```

Code 8: ผลลัพธ์และสถานะจาก ps ในกรณี 2.2

Table 2: สรุปค่าที่สังเกตได้ในกรณี 2.2

| รายการ                           | ค่าที่สังเกตได้            |
|----------------------------------|----------------------------|
| PID ของ parent                   | 6857                       |
| PID ของ child                    | 6859                       |
| PPID ของ child (ก่อน parent ตาย) | 6857 (parent ตัวจริง)      |
| PPID ของ child (หลัง parent ตาย) | <b>6846</b> (ถูก reparent) |
| สถานะของ child ใน ps             | S+ (ปกติ ไม่ใช่ zombie)    |

### ตรวจสอบเพิ่มเติมว่า PID 6846 คือ process ใด

```
$ ps -p 6846 -o pid=,ppid=,comm=,args=
6846   6844 Relay(6848)  /init

$ # walk the ancestry chain up to PID 1
PID    PPID COMMAND
6846   6844 Relay(6848)      <- WSL /init relay : the adopting subreaper
6844     2 SessionLeader
2       1 init-systemd(Ub
1       0 systemd      <- the real PID 1
```

Code 9: สายบรรพบุรุษของ process ที่รับ orphan ไปดูแล

ผลลัพธ์นี้แสดงว่า child **ไม่ได้** ถูกยกให้ PID 1 โดยตรงตามที่มักเข้าใจกัน แต่ถูกยกให้กระบวนการ `/init` ของ WSL ซึ่งลงทะเบียนตัวเองเป็น *child subreaper* ไปด้วย `prctl(PR_SET_CHILD_SUBREAPER)` เพราะกฎของเคอร์เนลคือยก orphan ให้ **subreaper ที่ใกล้ที่สุด** ไม่ใช่ยกให้ PID 1 เสมอไป บนระบบ Linux ที่ติดตั้งแบบ native หรือใน VM ทั่วไป คำนี้ออกมาเป็น 1 หรือเป็น PID ของ `systemd --user`

## 2.3 อภิปรายผล

จากการทดลองควบคุมลำดับการจบการทำงานของกระบวนการด้วย `sleep()` ในข้อ 2.1 และ 2.2 สรุปข้อแตกต่างและประเด็นสำคัญได้ดังนี้

- **ข้อ 2.1 ปรากฏการณ์ Zombie Process:** เมื่อ Child ตายก่อนแต่ Parent ยังทำงานอยู่โดยไม่ได้เรียก `wait()` เคอร์เนลจะคืนหน่วยความจำของ Child ทันที (`SZ = 0`) แต่ยังคงจอง PID และ Process Table Entry ค้างไว้ ทำให้เกิดสถานะ Z หรือ `<defunct>` จนกว่า Parent จะจบโปรแกรมหรือเรียก `wait()` เพื่ออ่าน Exit Status
- **ข้อ 2.2 ปรากฏการณ์ Orphan Process และการ Reparent:** เมื่อ Parent ตายก่อนโดยที่ Child ยังคงทำงานอยู่ Child จะกลายเป็นกระบวนการกำพร้า (Orphan) เคอร์เนลจะยก Child ให้กับ Sub-reaper ที่ใกล้ที่สุด (เช่น `systemd` หรือ `/init` ของ WSL) เป็น Parent คนใหม่ทันที สังเกตได้จากค่า PPID ของ Child ที่เปลี่ยนไปในการเรียก `getppid()` ครั้งที่สอง
- **ความแตกต่างสำคัญและปรากฏการณ์ `<defunct>`:** Zombie (`<defunct>`) คือกระบวนการที่สิ้นสุดการทำงานแล้วแต่ยังถูกจองค้างในตารางกระบวนการ หากสะสมมากจะสิ้นเปลือง PID และทำให้สร้าง Process ใหม่ไม่ได้ ต่างจาก Orphan ที่เป็นกระบวนการซึ่งยังคงทำงานตามปกติ และเมื่อ Orphan สิ้นสุดการทำงาน Parent คนใหม่ (Subreaper) จะทำหน้าที่เรียก `wait()` เก็บกวาดให้อัตโนมัติโดยไม่เกิด Zombie ทิ้งไว้

### 3 จำนวน process สูงสุดที่ fork ได้

การทดลองนี้ต้องการหาว่าระบบยอมให้เรียก `fork()` ได้สำเร็จกี่ครั้ง โดยใช้โครงสร้างแบบ โซ่ (chain) ตามที่ Hint ของโจทย์แนะนำ คือทุก process สร้างลูกเพียงตัวเดียวแล้วรอด้วย `waitpid()` โซ่จึงยาวขึ้นทีละหนึ่งและคลายตัว กลับจากปลายสุดอย่างเป็นระเบียบ ซึ่ง **ไม่ใช่** fork bomb และหยุดได้ทุกเมื่อ

#### 3.1 Source code

##### 3.1.1 ส่วนหลัก: ฟังก์ชัน `chain()` และการรายงานผล

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <errno.h>
5  #include <unistd.h>
6  #include <sys/wait.h>
7  #include <sys/resource.h>
8
9  #define HARD_CAP 200000          /* never build a chain longer than this */
10
11 struct result {
12     int count;                  /* number of successful fork() calls */
13     int err;                    /* errno that stopped us (0 = hit HARD_CAP) */
14 };
15
16 static int pfd[2];
17
18 /* Send the tally up the chain to the root and leave. */
19 static _Noreturn void report(int count, int err)
20 {
21     struct result r;
22     r.count = count;
23     r.err = err;
24     if (write(pfd[1], &r, sizeof r) != (ssize_t)sizeof r)
25         _exit(1);
26     _exit(0);
27 }
28
29 #pragma GCC diagnostic push
30 #pragma GCC diagnostic ignored "-Winfinite-recursion"
31
32 /* I am the process produced by fork() call number `level`. */
33 static void chain(int level)
34 {
35     pid_t pid;
36
37     if (level >= HARD_CAP)
38         report(level, 0);      /* stopped by our own ceiling */
39
40     pid = fork();
41
42     if (pid < 0)
43         /* fork #(level+1) failed -> `level` forks succeeded in total */
44         report(level, errno);
45

```

```
46     if (pid == 0)
47         chain(level + 1);      /* the child keeps digging */
48
49     /* parent of this link: hold still until the chain below me unwinds */
50     waitpid(pid, NULL, 0);
51     _exit(0);
52 }
53
54 #pragma GCC diagnostic pop
```

Code 10: q3\_forkcount.c – the chain and the reporting path

**คำอธิบายการทำงาน:** `chain(level)` หมายถึง “ฉันคือ process ที่เกิดจาก `fork()` ครั้งที่ `level`” แต่ละตัวจะ `fork()` หนึ่งครั้ง ถ้าสำเร็จ ลูกจะเรียก `chain(level+1)` ขุดลึกต่อไป ส่วนพ่อจะติดอยู่ที่ `waitpid()` ไม่ทำอะไรจนกว่าโซ่ข้างล่างจะคลายกลับมา ทำให้มีเพียง process เดียวเท่านั้นที่ทำงานอยู่ในแต่ละขณะ เมื่อ `fork()` ล้มเหลว process ที่ลึกที่สุดจะส่งผลรวมและ `errno` กลับไปให้ root ผ่าน pipe ด้วย `report()` ส่วน `HARD_CAP` เป็นเพดานของโปรแกรมเองไว้กันพลาด และคำสั่ง `#pragma` จำเป็นต้องมีเพราะ gcc มองว่า `chain()` เป็น infinite recursion ซึ่งเป็นการเตือนที่ผิดพลาด เนื่องจากการเรียกซ้ำเกิดใน process ใหม่คนละตัวและหยุดเมื่อ `fork()` ล้มเหลว

### 3.1.2 ส่วน main(): กำหนดเพดานและรับผลลัพธ์

```

1  int main(int argc, char *argv[])
2  {
3      pid_t pid;
4      struct result r = { 0, 0 };
5      struct rlimit rl;
6
7      if (argc > 1) {          /* optional artificial ceiling for rehearsal */
8          rlim_t want = (rlim_t)strtoul(argv[1], NULL, 10);
9          if (getrlimit(RLIMIT_NPROC, &rl) == 0) {
10             rl.rlim_cur = want;
11             if (rl.rlim_max != RLIM_INFINITY && rl.rlim_cur > rl.rlim_max)
12                 rl.rlim_cur = rl.rlim_max;
13             if (setrlimit(RLIMIT_NPROC, &rl) != 0)
14                 perror("setrlimit");
15         }
16     }
17
18     if (getrlimit(RLIMIT_NPROC, &rl) == 0)
19         printf("RLIMIT_NPROC in effect: soft = %ld, hard = %ld\n",
20             (long)rl.rlim_cur, (long)rl.rlim_max);
21
22     printf("Root PID = %d. Forking a chain until fork() fails ...\n", getpid());
23     fflush(stdout);
24
25     if (pipe(pfd) < 0) {
26         perror("pipe");
27         exit(EXIT_FAILURE);
28     }
29
30     pid = fork();
31     if (pid < 0) {
32         r.count = 0;
33         r.err = errno;
34     } else if (pid == 0) {
35         close(pfd[0]);
36         chain(1);          /* this process came from fork() #1 */
37         _exit(0);          /* unreachable */
38     } else {
39         close(pfd[1]);      /* only the descendants write */
40         waitpid(pid, NULL, 0);
41         if (read(pfd[0], &r, sizeof r) != (ssize_t)sizeof r) {
42             fprintf(stderr, "could not read result from pipe\n");
43             exit(EXIT_FAILURE);
44         }
45         close(pfd[0]);
46     }
47
48     printf("fork() succeeded %d times.\n", r.count);
49     if (r.err == 0)
50         printf("Stopped by the program's own HARD_CAP (%d), not by the OS.\n",
51             HARD_CAP);
52     else
53         printf("fork() #%d failed with errno %d (%s).\n",
54             r.count + 1, r.err, strerror(r.err));
55
56     return 0;
57 }

```

---

Code 11: q3\_forkcount.c – main(): optional ceiling and result collection

**คำอธิบายการทำงาน:** main() สร้าง pipe ก่อนแล้วจึง fork() เพื่อเริ่มใช้ ฝั่ง root ปิดปลายเขียน (pfd[1]) เพราะมีแต่ลูกหลานที่จะเขียน ส่วนลูกตัวแรกปิดปลายอ่าน (pfd[0]) การที่ root เรียก waitpid() ก่อนแล้วค่อย read() ไม่ทำให้เกิด deadlock เพราะข้อมูลขนาด 8 ไบต์ถูกเขียนค้างไว้ใน buffer ของ pipe ตั้งแต่ตอนที่ process ปลายสุดยังมีชีวิตอยู่ และเนื่องจากขนาดน้อยกว่า PIPE\_BUF การเขียนจึงเป็น atomic ไม่มีทางถูกแทรก ส่วนอาร์กิวเมนต์ argv[1] เป็นทางเลือกให้ตั้งเพดานจำลองด้วย setrlimit() เพื่อซ้อมการทดลองในระดับที่ปลอดภัยก่อนรันจริง

## 3.2 ผลการทดลอง

ทำการทดลอง 2 ชุดด้วยเพดานต่างกัน เพื่อยืนยันว่าข้อสรุปไม่ได้ขึ้นกับขนาดของเพดาน ชุด A ใช้ `ulimit -u 300` ซึ่งรันเร็ว ส่วนชุด B ใช้ `ulimit -u 6000` ซึ่งใกล้เคียงสถานการณ์จริงมากกว่า

### 3.2.1 3.1 การวัดครั้งแรก

```
$ gcc -Wall -Wextra -o q3_forkcount q3_forkcount.c

--- Set A ---
$ ulimit -u 300
$ ./q3_forkcount
RLIMIT_NPROC in effect: soft = 300, hard = 300
Root PID = 25252. Forking a chain until fork() fails ...
fork() succeeded 255 times.
fork() #256 failed with errno 11 (Resource temporarily unavailable).

--- Set B ---
$ ulimit -u 6000
$ ./q3_forkcount
RLIMIT_NPROC in effect: soft = 6000, hard = 6000
Root PID = 6921. Forking a chain until fork() fails ...
fork() succeeded 5953 times.
fork() #5954 failed with errno 11 (Resource temporarily unavailable).
```

Code 12: ผลลัพธ์การวัดครั้งแรกของทั้งสองชุด

### 3.2.2 3.2 เมื่อมีโปรแกรมอื่นทำงานค้างอยู่

ชุด B ทำการวัดต่อเนื่อง 3 เฟสภายใต้เงื่อนไขเดียวกัน โดยตรวจนับจำนวน thread ของผู้ใช้ก่อนวัดทุกครั้ง เพื่อยืนยันว่าสภาพระบบกลับสู่จุดเริ่มต้นจริง

```
===== PHASE 1 : before starting any extra program =====
UID threads now: 8
$ ./q3_forkcount
fork() succeeded 5953 times.
fork() #5954 failed with errno 11 (Resource temporarily unavailable).

===== PHASE 2 : with 100 extra programs running =====
$ for i in $(seq 1 100); do sleep 900 & done
UID threads now: 108
$ ./q3_forkcount
fork() succeeded 5853 times.                <- exactly 100 fewer
fork() #5854 failed with errno 11 (Resource temporarily unavailable).

===== PHASE 3 : after stopping all extra programs =====
$ kill $PIDS
UID threads now: 8
$ ./q3_forkcount
fork() succeeded 5953 times.                <- back to the original value
fork() #5954 failed with errno 11 (Resource temporarily unavailable).
```

Code 13: ผลลัพธ์ทั้งสามเฟสของชุด B (ulimit -u 6000)



Table 3: เปรียบเทียบจำนวน `fork()` ที่สำเร็จในแต่ละสถานการณ์

| ชุด            | สถานการณ์              | process อื่นที่เพิ่ม | <code>fork()</code> สำเร็จ (ครั้ง) |
|----------------|------------------------|----------------------|------------------------------------|
| A (เพดาน 300)  | ก่อนรันโปรแกรมเพิ่ม    | –                    | 255                                |
| A (เพดาน 300)  | ระหว่างรันโปรแกรมเพิ่ม | 20                   | 235                                |
| A (เพดาน 300)  | หลังปิดโปรแกรมทั้งหมด  | –                    | 256                                |
| B (เพดาน 6000) | ก่อนรันโปรแกรมเพิ่ม    | –                    | 5953                               |
| B (เพดาน 6000) | ระหว่างรันโปรแกรมเพิ่ม | 100                  | 5853                               |
| B (เพดาน 6000) | หลังปิดโปรแกรมทั้งหมด  | –                    | 5953                               |

### 3.3 อภิปรายผล

จากการทดลองวัดจำนวน `fork()` ที่สำเร็จภายใต้สภาพระบบที่ต่างกัน สรุปประเด็นสำคัญได้ดังนี้

- **ขีดจำกัดที่แท้จริงคือ `RLIMIT_NPROC`:** `fork()` return `-1` พร้อม `errno = 11` ซึ่งคือ `EAGAIN` (Resource temporarily unavailable) เหมือนกันทุกครั้งในทุกการทดลอง ไม่ใช่ `ENOMEM` แสดงว่าสิ่งที่หยุดเราคือโควตาจำนวน process ต่อผู้ใช้ ไม่ใช่หน่วยความจำหรือขนาดของตารางกระบวนการ
- **จำนวนที่ได้ไม่เท่ากับเพดานเพราะโควตาถูกใช้ไปก่อนแล้ว:** เพดาน 300 ได้ 255 ครั้ง (ส่วนต่าง 45) และเพดาน 6000 ได้ 5953 ครั้ง (ส่วนต่าง 47) เนื่องจาก `RLIMIT_NPROC` นับ process ของ UID นั้น ทั้ง user namespace ไม่ใช่แค่เฉพาะที่โปรแกรมเราสร้าง ส่วนต่างจึงคือ process ที่ผู้ใช้มีอยู่ก่อนแล้ว เช่น shell, WSL relay และ session อื่น ที่เปิดค้างไว้ สังเกตว่า `ps` มองเห็นเพียง 8 ตัวใน session ปัจจุบัน แต่ถูกหักไป 47 แสดงว่ามีอีกราว 39 ตัวที่อยู่นอก session ปัจจุบัน แต่ยังถูกนับรวมในโควตาเดียวกัน
- **จำนวนลดลงตามจำนวนโปรแกรมที่รันค้างแบบหนึ่งต่อหนึ่ง:** เมื่อเปิดโปรแกรม `sleep` ค้างไว้ 100 ตัว จำนวน `fork()` ที่สำเร็จลดลงจาก 5953 เหลือ 5853 คือลดลง **100 พอดีเป๊ะ** ไม่ขาดไม่เกิน เช่นเดียวกับชุด A ที่เปิด 20 ตัวแล้วลดลง 20 พอดี ยืนยันว่าโควตานี้เป็นทรัพยากรที่แชร์กันทั้งผู้ใช้ ไม่ใช่โควตาต่อโปรแกรม
- **ค่าจะกลับมาเท่าเดิมก็ต่อเมื่อควบคุมตัวแปรได้ครบ:** ชุด B กลับมาที่ 5953 เท่าเดิมเป๊ะ เพราะปิดเฉพาะ PID ที่เปิดเองและตรวจนับ thread ยืนยันว่ากลับมาที่ 8 เท่าเดิมก่อนวัดซ้ำ ในขณะที่ชุด A ได้ 256 ครั้ง ซึ่งมากกว่าเดิม 1 เพราะระหว่างการทดลองมี process อื่นของผู้ใช้จบไปพอดี ทำให้โควตาว่างเพิ่มขึ้น จึงสรุปได้ว่าตัวเลขที่วัดได้เป็นค่าที่ขึ้นกับสภาพ ของระบบ ณ ขณะนั้น ไม่ใช่ค่าคงที่ประจำเครื่อง
- **ต้นทุนของโครงสร้างแบบ recursive chain:** การวัดแต่ละครั้งในชุด B ใช้เวลา 755–895 วินาที และใช้หน่วยความจำสูงสุด ราว 13 GB ที่ความยาวโซ่ประมาณ 5,900 process เพราะ process ที่ระดับ  $N$  มี stack frame สะสม  $N$  เฟรม ทุกครั้งที่ `fork()` เคอร์เนลต้องคัดลอก page table ที่ใหญ่ขึ้นเรื่อย ๆ ทำให้ทั้งเวลาและหน่วยความจำโตแบบ  $O(N^2)$  นี่คือเหตุผลที่ไม่สามารถรันจนถึงเพดานจริง 62,827 ได้บนเครื่องที่มี หน่วยความจำ 15.7 GB และเป็นเหตุผลที่ต้องกำหนด `ulimit -u` ให้ต่ำลงก่อนการทดลองทุกครั้ง

## 4 การสื่อสารผ่าน Pipe

โจทย์กำหนดให้สร้าง pipe ด้วย `pipe()` แล้ว `fork()` โดยให้ **child** เป็นผู้ส่ง และ **parent** เป็นผู้รับ ผู้ส่งต้องปิดปลายอ่านและผู้รับต้องปิด ปลายเขียนก่อนเริ่มสื่อสาร ผลลัพธ์ที่ต้องได้มีรูปแบบดังนี้

```
Child: Child PID: xxxxxx
Parent: Parent PID: yyyyyy
Parent: Hello from child PID: xxxxxx
```

### 4.1 Source code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/wait.h>
6
7 #define BUFSZ 128
8
9 int main(void)
10 {
11     int fd[2];
12     pid_t pid;
13     char msg[BUFSZ], buf[BUFSZ];
14     ssize_t n;
15
16     if (pipe(fd) < 0) {
17         perror("pipe");
18         exit(EXIT_FAILURE);
19     }
20
21     pid = fork();
22     if (pid < 0) {
23         perror("fork");
24         exit(EXIT_FAILURE);
25     }
26
27     if (pid == 0) {
28         /* ----- CHILD = SENDER ----- */
29         close(fd[0]); /* sender never reads */
30
31         printf("Child: Child PID: %d\n", getpid());
32         fflush(stdout);
33
34         snprintf(msg, sizeof msg, "Hello from child PID: %d", getpid());
35
36         if (write(fd[1], msg, strlen(msg) + 1) < 0) { /* +1 sends the '\0' */
37             perror("child write");
38             exit(EXIT_FAILURE);
39         }
40
41         close(fd[1]); /* signal EOF to the reader */
42         exit(EXIT_SUCCESS);
43     }
44
45     /* ----- PARENT = RECEIVER ----- */
```

```

46     close(fd[1]);                               /* receiver never writes */
47
48     /* Read FIRST, then print. The child flushes its own line to stdout before
49     * it writes into the pipe, so this read() cannot return until the child's
50     * line has already reached the terminal. That makes the output order
51     * deterministic and equal to the one the problem sheet specifies, using
52     * the blocking read as the synchronisation point -- no sleep() needed.
53     * Printing before the read would race the child and, in practice, put the
54     * parent's line first about 39 times out of 40. */
55     n = read(fd[0], buf, sizeof buf - 1);      /* blocks until the child writes */
56     if (n < 0) {
57         perror("parent read");
58         exit(EXIT_FAILURE);
59     }
60     buf[n] = '\0';
61
62     printf("Parent: Parent PID: %d\n", getpid());
63     printf("Parent: %s\n", buf);
64
65     close(fd[0]);
66     wait(NULL);                                /* reap the child */
67     return 0;
68 }

```

Code 14: q4\_pipe.c – child เป็นผู้ส่ง parent เป็นผู้รับ

**คำอธิบายการทำงาน:** โปรแกรมเรียก `pipe(fd)` ก่อน `fork()` เสมอ เพื่อให้ทั้งสอง process ได้รับสำเนาของ file descriptor ทั้งคู่ไป หลัง `fork()` จึงมี descriptor ชี้ไปที่ pipe เดียวกันถึงสี่ตัว แต่ละฝ่ายจึงต้องปิดปลายที่ตนไม่ใช้ทันที คือ child ปิด `fd[0]` เพราะเป็นผู้ส่งไม่ต้องอ่าน และ parent ปิด `fd[1]` เพราะเป็นผู้รับไม่ต้องเขียน จากนั้น child พิมพ์ PID ของตนเอง สร้างข้อความด้วย `snprintf()` แล้ว `write()` ลง pipe โดยส่งความยาว `strlen(msg)+1` เพื่อให้รวมอักขระ `'\0'` ไปด้วย ส่วน parent เรียก `read()` ซึ่งจะ block รอจนกว่าจะมีข้อมูล แล้วจึงพิมพ์ PID ของตนเองตามด้วยข้อความที่ได้รับ และปิดท้ายด้วย `wait(NULL)` เพื่อเก็บกวาด child ไม่ให้เหลือ zombie

## 4.2 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q4_pipe q4_pipe.c
$ ./q4_pipe
Child: Child PID: 28274
Parent: Parent PID: 28273
Parent: Hello from child PID: 28274

$ ./q4_pipe
Child: Child PID: 29279
Parent: Parent PID: 29277
Parent: Hello from child PID: 29279

$ ./q4_pipe
Child: Child PID: 29282
Parent: Parent PID: 29280
Parent: Hello from child PID: 29282

--- redirected to a file (buffering check) ---
$ ./q4_pipe > out.txt
$ cat out.txt
Child: Child PID: 29293
Parent: Parent PID: 29292
Parent: Hello from child PID: 29293
```

Code 15: ผลลัพธ์ของข้อ 4 (รันซ้ำหลายครั้ง)

Table 4: ผลการตรวจสอบความถูกต้องของข้อ 4

| รายการที่ตรวจ                                       | ผล                 |
|-----------------------------------------------------|--------------------|
| คอมไพล์ด้วย gcc -Wall -Wextra                       | ผ่าน ไม่มี warning |
| รูปแบบ output ตรงกับที่โจทย์กำหนด                   | ตรงทุกบรรทัด       |
| ลำดับ output ถูกต้อง (ทดสอบ 200 ครั้ง)              | 200 / 200          |
| PID ในข้อความที่รับ ตรงกับ PID ของ child (10 ครั้ง) | 10 / 10            |
| process ตกค้างหรือ zombie                           | 0 ตัว              |

## 4.3 อภิปรายผล

จากการทดลองสื่อสารระหว่าง process ผ่าน pipe สรุปประเด็นสำคัญได้ดังนี้

- **เหตุผลที่ file descriptor ถูกสืบทอดไปยัง child ได้:** fork() คัดลอกตาราง file descriptor ของ parent ไปให้ child ทั้งชุด ดังนั้นการเรียก pipe() ก่อน fork() จึงทำให้ทั้งสอง process มีทางเข้าถึง pipe เดียวกัน นี่คือเหตุผลที่ pipe ใช้สื่อสารได้เฉพาะ ระหว่าง process ที่มีความสัมพันธ์เป็นเครือญาติกันเท่านั้น ถ้าเรียก pipe() หลัง fork() จะได้ pipe คนละเส้นและคุยกันไม่ได้
- **เหตุผลที่ต้องปิดปลายที่ไม่ใช้:** หลัง fork() จะมี descriptor ชี้ไปที่ pipe เดียวกันถึงสี่ตัว หากผู้รับไม่ปิดปลายเขียนของตนเอง read() จะไม่มีวันได้รับ EOF เพราะเคอร์เนลจะรายงาน EOF ก็ต่อเมื่อปลายเขียน ทุกตัว ในระบบถูกปิดแล้วเท่านั้น โปรแกรมจะค้างรออยู่ตลอดไป ในทางกลับกันถ้าผู้ส่งไม่ปิดปลาย

อ่าน ก็จะสิ้นเปลือง descriptor และเสียอ่านข้อความของตัวเองกลับมาโดยไม่ตั้งใจ ซึ่งเป็นกรณีที่จะทดลองในข้อ 5

- **พฤติกรรม blocking ของ read():** เมื่อ pipe ยังว่าง read() จะพา process เข้าสู่สถานะ blocked แล้วปล่อย CPU ให้ผู้อื่น จนกว่าจะมีข้อมูลเข้ามาหรือปลายเขียนถูกปิดหมด จึงไม่ใช่การวนรอแบบสิ้นเปลือง CPU (busy waiting) และคุณสมบัตินี้เอง ที่ทำให้ pipe ทำหน้าที่เป็นกลไกซิงโครไนซ์ได้ในตัว
- **ปัญหาลำดับ output ที่พบและวิธีแก้:** โค้ดฉบับแรกให้ parent พิมพ์บรรทัดของตนเองก่อนแล้วค่อยเรียก read() ซึ่งไม่มีอะไรรับประกันว่า child จะได้ CPU ก่อน ผลการทดสอบ 40 ครั้งพบว่า บรรทัดแรกเป็นของ parent ถึง 39 ครั้ง ทำให้สองบรรทัดแรกสลับกัน ไม่ตรงกับรูปแบบที่โจทย์กำหนด แก้โดยย้าย read() ขึ้นมาไว้ก่อน printf() ของ parent ซึ่งสร้างความสัมพันธ์แบบ happens-before ต่อกันเป็นลูกโซ่ คือ child ต้อง fflush() บรรทัดของตนออกหน้าจอก่อน จึงจะ write() ลง pipe และ read() ของ parent จะคืนค่าได้ ก็ต่อเมื่อการเขียนนั้นเกิดขึ้นแล้ว บรรทัดของ child จึงต้องปรากฏก่อน บรรทัดของ parent เสมอ ไม่ใช่เพียงแค่มีโอกาสสูง วิธีนี้ใช้ blocking read เป็นจุดซิงโครไนซ์ในตัว จึงดีกว่าการใส่ sleep() ซึ่งเป็นเพียงการเดาเวลาและยังคงมีโอกาสผิดพลาด ผลลัพธ์ก็คือลำดับถูกต้อง 200 จาก 200 ครั้ง
- **ไม่พบปัญหา buffer ข้างเหมือนข้อ 1:** เมื่อ redirect output ลงไฟล์ ผลลัพธ์ยังคงถูกต้องครบสามบรรทัดไม่มีข้อความซ้ำ เพราะโปรแกรมนี้ไม่มีการเรียก printf() ก่อน fork() buffer ของ stdout จึงว่างอยู่แล้วในขณะที่ทำสำเนา address space

## 5 การทดลองกรณีพิเศษของ Pipe

ข้อนี้ดัดแปลงโปรแกรมข้อ 4 ออกเป็นสามเวอร์ชัน เพื่อทดสอบพฤติกรรมของ pipe ในสถานการณ์ที่ผิดไปจากการใช้งานปกติ เนื่องจากโค้ดทั้งสามมีโครงสร้างหลัก เหมือนข้อ 4 ในรายงานนี้จึงแสดงเฉพาะ **ส่วนที่แก้ไข** ของแต่ละเวอร์ชัน โปรแกรมทุกตัวมีกลไกกันการค้างถาวรไว้ด้วย

### 5.1 5.1 ผู้ส่งพยายามอ่านข้อความของตัวเองกลับ

**วิธีทดลอง:** จงใจ *ไม่ปิด* descriptor ฝั่งใดเลย ทำให้ child ถือทั้งปลายอ่านและปลายเขียน จากนั้นให้ child เขียนข้อความลง pipe แล้วรีบ `read()` กลับทันที ส่วน parent ซึ่งเป็นผู้รับตัวจริงหน่วงด้วย `sleep(1)` ก่อนจึงอ่าน และตั้ง `alarm(3)` ไว้เป็นตาข่ายกันโปรแกรมค้างถาวร

```

1  if (pid == 0) {
2      /* ----- CHILD = SENDER, but it also reads ----- */
3      /* NOTE: deliberately NOT closing fd[0] -- that is the whole point. */
4      snprintf(msg, sizeof msg, "Hello from child PID: %d", getpid());
5      write(fd[1], msg, strlen(msg) + 1);
6
7      n = read(fd[0], buf, sizeof buf - 1);      /* read back its OWN pipe */
8      if (n > 0) {
9          buf[n] = '\0';
10         printf("[child ] read back %zd bytes: \"%s\"\n", n, buf);
11     }
12     exit(EXIT_SUCCESS);
13 }
14
15 /* ----- PARENT = the intended RECEIVER ----- */
16 sa.sa_handler = on_alarm;
17 sa.sa_flags = 0;      /* no SA_RESTART: we WANT read() to fail */
18 sigaction(SIGALRM, &sa, NULL);
19
20 sleep(1);      /* Let the child write AND read first */
21
22 alarm(3);      /* safety net so we never hang */
23 n = read(fd[0], buf, sizeof buf - 1);
24 alarm(0);

```

Code 16: ส่วนของโค้ดที่แก้ไขในข้อ 5.1

```

$ ./q5_1_sender_reads_own
[child ] writing: "Hello from child PID: 29443"
[child ] now reading from fd[0] (its OWN pipe) ...
[child ] read back 28 bytes: "Hello from child PID: 29443"    <-- its own message!
[parent] now trying to read the message ...
[parent] read FAILED: Interrupted system call
[parent] -> the pipe was empty: the sender had already consumed its own message.

```

Code 17: ผลลัพธ์ของข้อ 5.1

**สิ่งที่สังเกตได้:** child อ่านข้อความของตัวเองกลับมาได้ครบทั้ง 28 ไบต์ โดยไม่มีอะไรห้ามเลย ผลคือ parent ซึ่งเป็นผู้รับตัวจริงไม่ได้รับอะไรเลย ข้อมูลหายไปจากระบบ และที่สำคัญคือ parent ไม่ได้รับ EOF แต่ค้างอยู่ใน `read()` จนกระทั่ง `alarm(3)` ยิง `SIGALRM` ทำให้ `read()` คืน `-1` พร้อม `errno = EINTR` ทดสอบซ้ำ 3 ครั้งได้ผลเหมือนกันทุกครั้ง

## 5.2 5.2 ผู้รับอ่านก่อนที่ผู้ส่งจะส่ง

วิธีทดลอง: ให้ parent เรียก `read()` ทันทีตั้งแต่ pipe ยังว่าง ส่วน child หน่วง `sleep(3)` ก่อนจึงเขียนเวลาที่ `read()` ใช้ไปด้วย `clock_gettime()` เพื่อพิสูจน์ว่า parent ถูกพักอยู่ในเคอร์เนลจริง จากนั้นในเฟสที่สองให้ parent อ่านซ้ำอีกครั้งหลังจาก child ปิดปลายเขียนและจบไปแล้ว

```

1  if (pid == 0) {
2      /* ----- CHILD = SENDER, but a slow one ----- */
3      close(fd[0]);
4      sleep(SEND_DELAY);          /* 3 seconds late on purpose */
5
6      snprintf(msg, sizeof msg, "Hello from child PID: %d", getpid());
7      write(fd[1], msg, strlen(msg) + 1);
8
9      close(fd[1]);              /* closing the last write end -> reader sees EOF */
10     exit(EXIT_SUCCESS);
11 }
12
13 /* ----- PARENT = RECEIVER, reads far too early ----- */
14 close(fd[1]);
15
16 t0 = now();                    /* phase 1: pipe is still empty */
17 n = read(fd[0], buf, sizeof buf - 1);
18 printf("[parent] read() returned after %.2f s with %zd bytes\n",
19        now() - t0, n);
20
21 t0 = now();                    /* phase 2: no writer left */
22 n = read(fd[0], buf, sizeof buf - 1);
23 printf("[parent] read() returned %zd after %.2f s\n", n, now() - t0);

```

Code 18: ส่วนของโค้ดที่แก้ไขในข้อ 5.2

```

$ ./q5_2_read_before_send
[parent] calling read() immediately, pipe is still empty ...
[child ] sleeping 3 s before sending ...
[child ] sending now: "Hello from child PID: 29447"
[child ] write end closed, exiting.
[parent] read() returned after 3.00 s with 28 bytes: "Hello from child PID: 29447"
[parent] -> read() BLOCKED, it did not fail and did not return 0.
[parent] reading again, now that every write end is closed ...
[parent] read() returned 0 after 0.00 s -> EOF (no writer left, so no blocking)

```

Code 19: ผลลัพธ์ของข้อ 5.2

สิ่งที่สังเกตได้: การเรียก `read()` บน pipe ที่ว่างไม่ได้ผิดพลาดและไม่ได้คืนค่า 0 แต่ **block รอ** เวลาที่วัดได้คือ 3.00 วินาที ตรงกับ `sleep()` ของฝั่งผู้ส่งพอดี ซึ่งพิสูจน์ว่า parent ถูกพักไว้ในสถานะ blocked จริง ไม่ได้วนรอแบบ busy waiting ส่วนในเฟสที่สองเมื่อปลายเขียนถูกปิดหมดทุกตัวแล้ว `read()` คืนค่า 0 คือ EOF ทันทีภายใน 0.00 วินาที ไม่ block อีกต่อไป ทดสอบซ้ำ 3 ครั้งได้เวลาตรงกันทุกครั้ง

### 5.3 5.3 ผู้ส่งส่งหลายข้อความก่อนที่ผู้รับจะอ่าน

วิธีทดลอง: เฟสที่หนึ่งให้ child เขียน 3 ข้อความรวดเดียวขณะที่ parent ยัง `sleep(1)` อยู่ แล้วให้ parent เรียก `read()` เพียงครั้งเดียวด้วย buffer ขนาดใหญ่ เฟสที่สองให้ child เขียนอีก 3 ข้อความ แล้วให้ parent อ่านทีละ 8 ไบต์ ใช้อักขระ | เป็นตัวคั่นข้อความเพื่อให้มองเห็นขอบเขตได้ด้วยตา

```

1  if (pid == 0) {
2      /* ----- CHILD = SENDER ----- */
3      close(fd[0]);
4
5      for (i = 1; i <= NMSG; i++) {      /* batch 1: three writes in a row */
6          snprintf(msg, sizeof msg, "MSG%d from child PID: %d|", i, getpid());
7          write(fd[1], msg, strlen(msg));      /* '|' is our delimiter */
8      }
9
10     sleep(2);      /* Let the parent finish phase 1 before phase 2 starts */
11
12     for (i = NMSG + 1; i <= 2 * NMSG; i++) {      /* batch 2 */
13         snprintf(msg, sizeof msg, "MSG%d|", i);
14         write(fd[1], msg, strlen(msg));
15     }
16     close(fd[1]);
17     exit(EXIT_SUCCESS);
18 }
19
20 /* ----- PARENT = RECEIVER, reads late ----- */
21 close(fd[1]);
22 sleep(1);      /* Let the child queue 3 messages */
23
24 n = read(fd[0], buf, sizeof buf - 1); /* PHASE 1: one big read() */
25 dump("[parent] got", buf, n);
26
27 sleep(2);
28      /* PHASE 2: 8 bytes at a time */
29 while ((n = read(fd[0], buf, SMALL)) > 0)
30     dump("[parent] chunk", buf, n);

```

Code 20: ส่วนของโค้ดที่แก้ไขในข้อ 5.3

```

$ ./q5_3_many_messages
[child ] write #1: "MSG1 from child PID: 29450|"
[child ] write #2: "MSG2 from child PID: 29450|"
[child ] write #3: "MSG3 from child PID: 29450|"
[parent] PHASE 1: one big read()
[parent] got 81 bytes: [MSG1 from child PID: 29450|MSG2 from child PID: 29450|MSG3 from
child PID: 29450|]
[parent] -> 3 separate write() calls arrived in ONE read(). Message boundaries are gone.
[child ] write #4: "MSG4|"
[child ] write #5: "MSG5|"
[child ] write #6: "MSG6|"
[parent] PHASE 2: reading 8 bytes at a time
[parent] chunk 8 bytes: [MSG4|MSG]      <- MSG5 is cut in half here
[parent] chunk 7 bytes: [5|MSG6|]      <- and continues here
[parent] read() returned 0 -> EOF, sender is gone.

```

Code 21: ผลลัพธ์ของข้อ 5.3



**สิ่งที่สังเกตได้:** ในเฟสที่หนึ่ง การเรียก `write()` 3 ครั้งถูกอ่านกลับมาได้ด้วย `read()` เพียงครั้งเดียวรวม 81 ไบต์ ( $27 \times 3$ ) ข้อความทั้งสามถูกต่อกันเป็นสายเดียวโดยไม่มีร่องรอยว่าแต่ละ `write()` เริ่มและจบตรงไหน ส่วนในเฟสที่สองเมื่ออ่านทีละ 8 ไบต์ ข้อความ MSG5 | ถูกผ่าครึ่ง โดยครึ่งแรกไปอยู่ที่ chunk ที่หนึ่ง และครึ่งหลังไปอยู่ที่ chunk ที่สอง อย่างไรก็ตามลำดับของข้อมูลยังคงถูกต้องแบบ FIFO เสมอ ทดสอบซ้ำ 5 ครั้งได้ผลเหมือนกันทุกครั้ง

## 5.4 สรุปผลการทดลองข้อ 5

Table 5: สรุปพฤติกรรมของ pipe ในแต่ละสถานการณ์

| สถานการณ์                   | ผลที่สังเกตได้                                                                                  | คำอธิบาย                                                                                               |
|-----------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 5.1 ผู้ส่งอ่านข้อความตัวเอง | ผู้ส่งได้ข้อความคืนครบ 28 ไบต์ ส่วนผู้รับตัวจริงไม่ได้อะไร และค้างจนถูก EINTR                   | pipe เป็นคิว FIFO ที่ข้อมูลถูกอ่านแล้วหมดไป ไม่ใช้การ broadcast ใครอ่านก่อนได้ก่อน                     |
| 5.2 ผู้รับอ่านก่อนผู้ส่ง    | read() block นาน 3.00 วินาที เฟสที่สองคืนค่า 0 ภายใน 0.00 วินาที                                | ว่างและยังไม่มีผู้เขียนอยู่จะ block ส่วนว่างและไม่มีผู้เขียนเหลือแล้ว จะได้ EOF                        |
| 5.3 ส่งหลายข้อความก่อนอ่าน  | write() 3 ครั้งถูกอ่านด้วย read() ครั้งเดียว 81 ไบต์ และเมื่ออ่านทีละ 8 ไบต์ ข้อความถูกผ่าครึ่ง | pipe เป็น byte stream ที่รักษาลำดับ แต่ไม่เก็บขอบเขตข้อความ การแบ่งข้อความ เป็น หน้าที่ ของแอปพลิเคชัน |

## 5.5 อภิปรายผล

จากการทดลองทั้งสามกรณี สรุปคุณสมบัติสำคัญของ pipe ที่ได้เรียนรู้ได้ดังนี้

- **pipe ไม่มีแนวคิดเรื่องผู้ส่งและผู้รับ:** ข้อ 5.1 แสดงว่าเคอร์เนลไม่ได้จดจำว่าใครเป็นผู้เขียนข้อมูลลง pipe เป็นเพียงคิวไบบทแบบ FIFO ที่ใครก็ตามซึ่งถือปลายอ่านอยู่สามารถ ดึงข้อมูลออกไปได้ และเมื่อถูกอ่านแล้วข้อมูลจะหายไปจากคิวทันที ไม่ได้ถูกส่งซ้ำให้ผู้อ่านคนอื่น การกำหนดบทบาทผู้ส่งกับผู้รับจึงต้องทำ ด้วยวินัยของโปรแกรมเมอร์เอง คือการปิดปลายที่ไม่ใช้ตามที่ทำในข้อ 4
- **การไม่ปิดปลายที่ไม่ใช้ทำให้ตรวจจบ EOF ไม่ได้:** ในข้อ 5.1 parent ค้างอยู่ใน read() แทนที่จะได้รับ EOF ทั้งที่ child จบไปแล้ว เพราะตัว parent เองยังถือปลายเขียนไว้ เคอร์เนลจะรายงาน EOF ก็ต่อเมื่อปลายเขียน **ทุกตัว** ในระบบถูกปิด เปรียบเทียบกับข้อ 5.2 ที่ปิดปลายที่ไม่ใช้อย่างถูกต้อง จะเห็นว่า read() คืนค่า 0 ได้ทันทีเมื่อผู้เขียนหมดไป
- **blocking read เป็นกลไกชิงโครไนซ์ในตัว:** ข้อ 5.2 วัดได้ว่า read() หยุดรอนาน 3.00 วินาที ตรงกับ sleep() ของผู้ส่งพอดี แสดงว่า process ถูกพักไว้ในสถานะ blocked และคืน CPU ให้ผู้อื่นใช้ ไม่ใช่การวนรอแบบ busy waiting คุณสมบัตินี้คือสิ่งเดียวกับที่นำไปใช้แก้ปัญหาลำดับ output ในข้อ 4
- **pipe เป็น byte stream ไม่ใช่ message queue:** ข้อ 5.3 แสดงชัดเจนสองทาง คือหลายข้อความถูกรวบรวมใน read() ครั้งเดียว และข้อความเดียวถูกแบ่งไปอยู่ใน read() หลายครั้ง pipe รับประกันเพียง ลำดับ ของไบบทว่าเป็น FIFO เท่านั้น แต่ไม่รับประกัน ขอบเขต ของข้อความ ดังนั้นการแบ่งข้อความ (framing) จึงเป็นหน้าที่ของแอปพลิเคชัน เช่น ใช้ระบุเป็นขนาดคงที่ ใส่ความยาวนำหน้า หรือใช้ตัวคั่นอย่าง '\0' และ '\n' ดังที่ข้อ 4 ส่ง strlen(msg)+1 เพื่อให้รวมตัวปิดสตริงไปด้วย
- **ความสำคัญของการมีกลไกกันโปรแกรมค้าง:** การทดลองข้อ 5.1 จะค้างถาวรถ้าไม่มี alarm(3) คอยขัดจังหวะ เพราะเป็นสถานการณ์ที่ผู้รับรอข้อมูลที่ไม่มีวันมาถึง การใส่ timeout ไว้จึงเป็นแนวปฏิบัติที่จำเป็นเมื่อทดลองกับ การสื่อสารแบบ blocking